

IMPROVING DEEP LEARNING OF ALPHA TERM STRUCTURES FROM THE ORDER BOOK

PETTER N. KOLM AND NICHOLAS WESTRAY

Petter N. Kolm is Clinical Professor and Director of the Mathematics in Finance Master's Program at NYU's Courant Institute of Mathematical Sciences, New York, NY 10012
petter.kolm@nyu.edu

Nicholas Westray is Visiting Researcher in Financial Machine Learning at NYU's Courant Institute of Mathematical Sciences, New York, NY 10012
nicholas.westray@nyu.edu

ABSTRACT. In recent years, deep learning (DL) models have experienced notable success in predicting high-frequency returns in equities by leveraging extensive order book data and directly extracting features from it. This marks a notable departure from current industry practice, where features are often manually crafted.

In this article, we address several practical open questions in this area, such as determining the most suitable network architecture and input selection for forecasting returns at multiple horizons (e.g. alpha term structures), optimizing the width and depth of neural network components to enhance performance, defining the appropriate historical data window size, and evaluating the benefits of incorporating time as a feature in the models. We evaluate the effectiveness of four DL models in forecasting high-frequency alpha term structures across various settings: a simple LSTM, a multi-head LSTM, an LSTM Seq2Seq without attention, and an LSTM Seq2Seq with attention. We find that surpassing the performance of a simple LSTM in the return forecasting task is surprisingly challenging.

Date: March 23, 2024.

Key words and phrases. Alpha Term Structures, Artificial Neural Networks, Deep Learning, Financial Machine Learning, High-Frequency Trading, Limit Order Books, Market Microstructure, Order Flow Imbalance, Return Predictability.

1. INTRODUCTION

Systematic trading in equities at intraday and higher frequencies is fiercely competitive, with alpha generation being essential for both buy- and sell-side firms. Sell-side entities seek to improve the performance of their next-generation trade execution algorithms, while buy-side firms aim at developing and deploying profitable high-frequency trading strategies. The current state of the art in the space typically involves employing quants to extract and handcraft features using expert domain knowledge in trading and market microstructure. Successful features and trading signals translate into high-value intellectual property for their respective firms.

Generating this kind of high-frequency alpha is exceptionally challenging for several reasons. First, it involves handling enormous amounts of data, often reaching terabyte and petabyte sizes, necessitating specialized infrastructure for storage, processing, and analysis. Second, the data itself is inherently noisy, non-stationary, and exhibits fat-tails. Furthermore, the space is highly competitive, with competitors striving to accomplish the same objectives using (almost) identical data sources. Undoubtedly, the “holy grail” for many of these firms is to use raw order book data, or a collection of order books, as inputs to an automated process capable of generating alpha forecasts.

In his essay “The Bitter Lesson,” Rich Sutton suggests where the holy grail might lie, as he writes¹

“The biggest lesson that can be read from 70 years of AI research is that general methods that leverage computation are ultimately the most effective, and by a large margin. The ultimate reason for this is Moore’s law, or rather its generalization of continued exponentially falling cost per unit of computation.”

“...researchers seek to leverage their human knowledge of the domain, but the only thing that matters in the long run is the leveraging of computation. These two need not run counter to each other, but in practice they tend to.”

Recent breakthroughs in several domains outside of finance underscore the importance of leveraging computation over domain-specific knowledge, bringing us closer to the holy grail in these fields. For instance, Deep Blue that defeated Kasparov in chess relied on extensive deep search algorithms rather than exploiting the special structure of the game itself (Newborn,

¹See, <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>.

2012). Similarly, in the game of Go, advancements in learning from self-play enabled the learning of value functions, giving a competitive advantage through the application of massive computation. (Silver et al., 2017). In computer vision, traditional techniques such as edge detection and cylinder recognition have been superseded by convolutional neural networks (CNNs) (LeCun et al., 1998; LeCun and Bengio, 1995). Likewise, in natural language processing, previous methods grounded in linguistics and knowledge of words and phonemes have given way to modern deep learning and statistically based approaches, such the BERT and GPT models (Devlin et al., 2018; Brown et al., 2020; Achiam et al., 2023).

Certainly, at this time there not many areas in finance where the type of *deep learning* (DL) techniques described above have resulted in such significant advancements. However, recent years have witnessed the success of DL models in forecasting high-frequency returns in equities by leveraging large limit order books (LOBs) (or simply order books, for short) data.² Specifically, a number of studies demonstrate that DL models outperform classical statistical and machine learning approaches, such as penalized linear models, decision trees, and kernel-based models, often by a significant margin. In most of these studies, model inputs are represented as raw or transformed time series of order book states and the return forecasting problem is cast as a *classification task*, where a single forecasting horizon is chosen to be either a deterministic time interval such as two seconds, or a stochastic time interval such as until the next price change.³ In many of the aforementioned studies, features (predictors) are learned directly from the data rather than being manually crafted, marking a significant departure from current industry practice.

Despite some consensus in the literature, several practical questions remain regarding network architecture and input selection. These include determining the best method for simultaneous forecasting of returns at multiple horizons (e.g. alpha term structures), optimizing the width and depth of neural network components to achieve superior performance, defining the

²See, for example, Tsantekidis et al. (2017), Tran et al. (2019), Zhang et al. (2018), Sirignano (2019), Sirignano and Cont (2019), Zhang et al. (2019a), Zhang et al. (2019b), Luo and Yu (2019), Fan et al. (2019), Tsantekidis et al. (2020), Briola et al. (2020), Zhang et al. (2021), Zhang and Zohren (2021), Lucchese et al. (2022), Kolm et al. (2023), Bilokon and Qiu (2023), Prata et al. (2023), and Briola et al. (2024).

³Notable exceptions include the work of Mäkinen et al. (2019), Rahimikia and Poon (2021), Zhang and Zohren (2021), and Kolm et al. (2023).

appropriate historical data window size, and assessing the advantages of incorporating time as a feature in the models. Further work is needed to address these questions and advance the comprehension and utilization of DL for high-frequency return forecasting.

In this article, we assess the effectiveness of four DNNs in forecasting high-frequency alpha term structures across various settings: (i) a simple LSTM, (ii) a multi-head LSTM, (iii) an LSTM Seq2Seq without attention, and (iv) an LSTM Seq2Seq with attention. Our empirical findings indicate that it is surprisingly hard to surpass the performance of a simple LSTM in this forecasting task. Specifically, we observe comparable performance among the LSTM, multi-head LSTM, and LSTM Seq2Seq models, while the attention-based LSTM Seq2Seq model exhibits inferior performance compared to the others.⁴ Therefore, with our focus on the LSTM for the rest of our empirical work, we present the following findings:

- Batch size: Although larger batch sizes result in more computationally efficient training by leveraging GPUs more effectively, smaller batch sizes, such as 64 and 256, have a meaningful impact on forecasting performance.
- Width vs. depth: Varying depth and width, we show there is a slight improvement with increased depth, albeit minimal. Keeping depth fixed, we observe a modest degradation in performance as width increases.
- Time features: Analyzing improvement in forecasting performance associated with different time features, we note that the combination of time of day, and the duration between LOB updates, shows the greatest improvement. Conversely, the LSTM model incorporating date features performs the worst, indicating the absence of significant seasonal effects.
- Lookback window size: Including longer-term information beyond a few hundred order book updates does not improve performance.
- Finally, we provide additional support to the findings of Kolm et al. (2023) that a more stable order book leads to stronger forecasting performance.

⁴We emphasize that the purpose of this article is not to provide an exhaustive comparison of architectures for mid-price return forecasting. Instead, we have selected some that provide a representative range of recent architectures for time series prediction, and several of them have been applied to similar forecasting problems.

The outline of the article is as follows. Section 2 provides a review of limit order book market mechanics and then describes the forecasting models we employ in this study. Section 3 presents our empirical results and findings, while Section 4 concludes. We share technical details regarding our deep learning models, data sources, data preprocessing, and the training and evaluation of our forecasting models in Appendix A and Appendix B.

2. METHODOLOGY

2.1. Limit Order Books and Order Flow Imbalance. Modern equity trading is primarily electronic on major global exchanges worldwide, facilitated by a *limit order book* (or *order book*, for short), one for each stock. The order book organizes buyers and sellers by price and time, allowing them to bid and offer stock for purchase or sale. Figure 1 illustrates the order book of a stock at a point in time t , with buyers depicted in blue and sellers in yellow. The highest price buyers are willing to pay for the stock is the *bid price* (b_t), while the lowest price sellers accept is the *ask price* (a_t). From the order book we obtain the *mid-price* ($p_t := (b_t + a_t)/2$), *bid-ask spread* ($a_t - b_t$), and *tick size*, the smallest price increment between price levels of the order book. In this article, we use the top ten non-zero levels of the order books of stocks on Nasdaq.

An *order* is defined by a four-tuple (side, price, quantity, time), indicating the side of the order book where the order is placed, the submitted price, the desired trading quantity, and the submission time. Orders can be entered and, if active, canceled at any time. Upon submission, the exchange's matching engine tries to pair it with existing orders in the order book. Matched orders are referred to as *market orders*, while unmatched or partially matched orders are known as *limit orders*. In cases where multiple limit orders share the same side and price, they are queued chronologically at that price level following the first-in-first-out (FIFO) principle. Gould et al. (2013), Abergel et al. (2016), and Bouchaud et al. (2018) provide a comprehensive account of modern equity trading and the mechanics of the order book.

We define the *state of the order book at time t* as the vector containing price and volume information for the top ten (non-empty) bid and ask levels

$$\mathbf{s}_t^{\text{LOB}} := (a_t^1, v_t^{1,a}, b_t^1, v_t^{1,b}, \dots, a_t^{10}, v_t^{10,a}, b_t^{10}, v_t^{10,b})^\top \in \mathbb{R}^{40}, \quad (1)$$

where b_t^i, a_t^i represent the bid and ask prices at the i -th level at time t , while $v_t^{i,b}, v_t^{i,a}$ denote the corresponding share volumes. The order book state of a

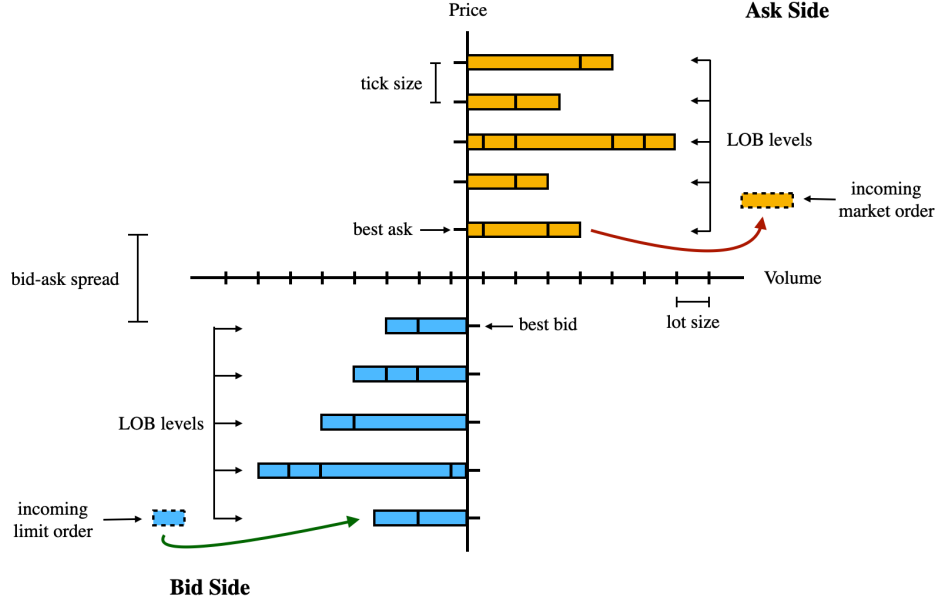


FIGURE 1. A market order executes on the ask side, and a new limit order is posted on the bid side of the stock's limit order book.

stock evolves as an irregularly spaced time series, continuously changing as buyers and sellers interact.

Letting $t \in \{1, \dots, T\}$ enumerate all order book states for a stock on a given day, we define the *bid order flows* (bOF) and *ask order flows* (aOF) at time t as the vectors $\mathbf{bOF}_t, \mathbf{aOF}_t \in \mathbb{R}^{10}$, with each component given by

$$\mathbf{bOF}_{t,i} := \begin{cases} v_t^{i,b}, & \text{if } b_t^i > b_{t-1}^i, \\ v_t^{i,b} - v_{t-1}^{i,b}, & \text{if } b_t^i = b_{t-1}^i, \\ -v_{t-1}^{i,b}, & \text{if } b_t^i < b_{t-1}^i, \end{cases} \quad (2)$$

$$\mathbf{aOF}_{t,i} := \begin{cases} -v_{t-1}^{i,a}, & \text{if } a_t^i > a_{t-1}^i, \\ v_t^{i,a} - v_{t-1}^{i,a}, & \text{if } a_t^i = a_{t-1}^i, \\ v_t^{i,a}, & \text{if } a_t^i < a_{t-1}^i, \end{cases} \quad (3)$$

for $i = 1, \dots, 10$. By concatenating and subtracting the bid and ask order flows at time t , we obtain the *order flow* (OF) and *order flow imbalance*

(OFI) via

$$\mathbf{OF}_t := \begin{pmatrix} \mathbf{bOF}_t \\ \mathbf{aOF}_t \end{pmatrix} \in \mathbb{R}^{20}, \quad (4)$$

$$\mathbf{OFI}_t := \mathbf{bOF}_t - \mathbf{aOF}_t \in \mathbb{R}^{10}. \quad (5)$$

The nonlinear transformations described by formulas (2), (3) and (5) represent a well-established approach for mapping nonstationary time series of order book states to stationary time series (Cont et al., 2014). In particular, Cont et al. (2014) establish a linear relationship between price changes and the OFI at the first level of the order book, and subsequent studies such as Xu et al. (2018), Cont et al. (2023), and Kolm and Westray (2023) extend this concept to multilevel and cross-sectional OFIs. The order flow, as defined in equation (4), preserves the separation between bid and ask sides, presenting a modest generalization of OFI. While in this article we focus on OFIs, OFs can provide more flexibility in some forecasting tasks (Kolm et al., 2023). In particular, unlike OFI which treats bid and ask order flows symmetrically, employing OFs allows forecasting models the possibility of combining them asymmetrically.

2.2. Forecasting Alpha Term Structures. We are interested in forecasting future mid-price stock returns across multiple horizons. The vector

$$\mathbf{r}_t := (r_{t,1}, \dots, r_{t,H})^\top \in \mathbb{R}^H, \quad \text{where } H \geq 1, \quad (6)$$

referred to as an *alpha term structure*, represents the multi-horizon forecasts for the stock and time t , where $H \geq 1$ is the number of horizons. We formulate our forecasting problem at the single stock level as the regression

$$\mathbf{r}_t = \mathbf{g}(\mathbf{OFI}_t, \mathbf{OFI}_{t-1}, \dots, \mathbf{OFI}_{t-W}) + \boldsymbol{\varepsilon}_t, \quad (7)$$

where the vector-valued function $\mathbf{g}$ is an ANN, $\boldsymbol{\varepsilon}_t \in \mathbb{R}^H$ is a residual, and W denotes the length of the lookback window. Unless explicitly noted otherwise, in our empirical work we adopt $W = 100$ for all stocks, as frequently used in related studies (see, for example, Zhang et al. (2019a) and Kolm et al. (2023)). To avoid issues around market open and close, we drop the first and last ten minutes of each day in our empirical work.

2.3. Deep Learning Models. We consider four DNNs for the regression-based supervised learning forecasting task in equation (7): (i) LSTM, (ii) multi-head LSTM, (iii) LSTM Seq2Seq without attention, and (iv) LSTM

Seq2Seq with attention. Table 1 provides a summary of their architectures. While these are standard in machine learning today, we present a short review of them below and refer to Appendix A.1 for additional details.

An LSTM model is a type of *recurrent neural network* (RNN) designed to tackle the issue of vanishing gradients in traditional RNNs (Hochreiter and Schmidhuber, 1997). LSTMs excel at processing sequential data by selectively remembering or forgetting information over long sequences. They consist of specialized memory cells and gates that control the flow of information, making them effective for tasks requiring long-term dependencies, such as time series analysis and natural language processing. In this article, we use the same LSTM as in Kolm et al. (2023).

In our previous work, we employed a single LSTM to embed the time series inputs, and then used its representation to generate mid-alpha term structure at multiple horizons (Kolm et al., 2023). However, there might be room to enhance forecasting performance by employing distinct LSTMs for each horizon, thereby leveraging customized embeddings. To examine this possibility, we incorporate a *multi-head LSTM* in this study, which is a variant of the single LSTM featuring multiple parallel processing units, also known as “heads.” Each head operates independently, allowing the model to capture different aspects of the input sequence simultaneously. This setup enables the model to learn more complex patterns and dependencies within the data compared to a standard LSTM. The key difference between a multi-head LSTM model and a single LSTM lies in the way information is processed. In a standard LSTM, there is a single set of parameters that governs how the input sequence is transformed and processed over time. In contrast, in a multi-head LSTM each head has its own set of parameters. This enables the multi-head LSTM to concurrently learn various representations of the input sequence, for example, by extracting information that is specific to forecasting returns at each horizon.

First introduced in Cho et al. (2014) and Sutskever et al. (2014), *sequence-to-sequence* (Seq2Seq) models are tailored for tasks with sequential data, taking sequential data as input and generating sequential data as output. Seq2Seq models have significantly improved the performance of *natural language processing* (NLP) and *machine translation* (MT) systems, which traditionally relied on classical statistical and phrase-based approaches. Seq2Seq models have two main components, an *encoder* and a *decoder*. The encoder processes an input sequence, such as a sequence of words in a sentence

Model	LSTM	multi-head LSTM	LSTM Seq2Seq	LSTM Seq2Seq + Attn
Layer Input	100×20	100×20	100×20	100×20
Hidden Layers	LSTM @ 256 Units (Tanh)	10 LSTMs @ 32 Units each (Tanh)	Encoder: LSTM @ 256 Units Decoder: LSTM @ 256 Units (Tanh)	Encoder: LSTM @ 256 Units Decoder: LSTM @ 256 Units (Tanh)
Layer Output	10×1	10×1	10×1	10×1

TABLE 1. The LSTM and Seq2Seq model architectures employed in this article to predict alpha term structures from OFIs.

for NLP tasks, or states of the order book, and converts it into a fixed-size context vector that represents the input sequence. The context vector encapsulates the information contained in the input sequence. Then, the decoder takes the context vector as input and generates an output sequence, which can vary in length. The output sequence could be a translation of the inputted sentence in the case of machine translation tasks, or an alpha term structure.

A *Seq2Seq with attention* (Seq2Seq + Attn) is an extension of the Seq2Seq model that incorporates an attention mechanism (Bahdanau et al., 2014). In a standard Seq2Seq model, the decoder receives only the final context vector from the encoder, which contains information about the entire input sequence. However, in attention-based Seq2Seq models, the decoder dynamically focuses on different parts of the input sequence at each step of the decoding process. This feature enables the attention-based Seq2Seq model to effectively handle long input sequences and produce more accurate outputs, particularly when certain segments of the inputs are more relevant. In contrast, a standard Seq2Seq model lacks the capability to assign varying weights to different input segments during decoding. In machine translation, text summarization, and speech recognition tasks, attention-based Seq2Seq models frequently outperform traditional Seq2Seq models (Rush et al., 2015; Chorowski et al., 2015; Luong et al., 2015).

Our implementation of the Seq2Seq models is adapted from Zhang and Zohren (2021), where we replace the classification layer with a dense layer.⁵ We employ LSTMs for both the encoders and decoders in the Seq2Seq and attention-based Seq2Seq models, referring to the resulting architectures as the *LSTM Seq2Seq* and *LSTM Seq2Seq + Attn* models.

We train and evaluate the models following a methodology similar to that of Kolm et al. (2023). We provide a brief overview and emphasize any distinctions in Appendix A.3.

3. EMPIRICAL RESULTS

In Section 3.1, we compare the performance of all the architectures mentioned above. In all other subsections, we exclusively use the LSTM.

3.1. Different Architectures. In Figure 2, we compare the out-of-sample forecasting accuracy of different models across various time horizons using

⁵The implementation of Zhang and Zohren (2021) is accessible at <https://github.com/zcakhay/Multi-Horizon-Forecasting-for-Limit-Order-Books>.

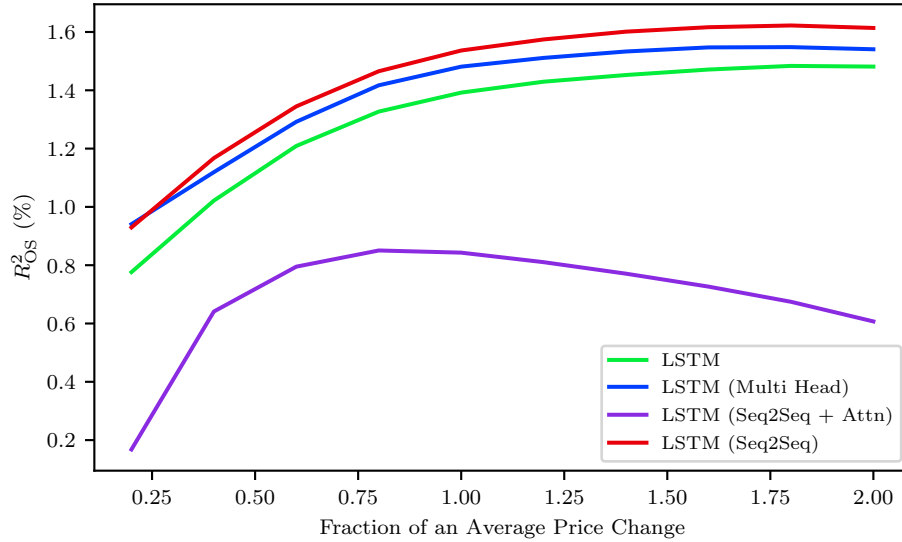


FIGURE 2. Out-of-sample forecasting performance of the models at different horizons from ten levels of order flows. Model performance is measured as the average R^2_{OS} across days and stocks. Horizons are given as the fraction of an average price change for each stock. Each model is trained and evaluated using a rolling-window out-of-sample methodology over thirteen months of data.

order flow from ten levels of the order book as inputs. Throughout this and following sections, we employ the rolling-window out-of-sample methodology spanning thirteen months, detailed in Appendix A.3. We calculate the average R^2_{OS} across both days and stocks to assess performance as in Kolm et al. (2023). Specifically, we first compute the daily average R^2_{OS} for each stock at each horizon. Subsequently, we aggregate the performance across stocks to derive the average R^2_{OS} for each model. This approach is effective for evaluating the overall aggregate performance of the forecasting models.

We observe that the LSTM, multi-head LSTM, and LSTM Seq2Seq have similar performance, while the attention-based LSTM Seq2Seq performs worse than the rest. These results suggest that the task of forecasting alpha term structures is quite different from that of machine translation and NLP where attention-based architectures excel.

Comparing our results to those in Kolm et al. (2023, Fig. 2), we observe an improvement in performance (with an R^2_{OS} of about 1.6% here compared to 1.3% in Kolm et al. (2023, Fig. 2)). This improvement is not attributable

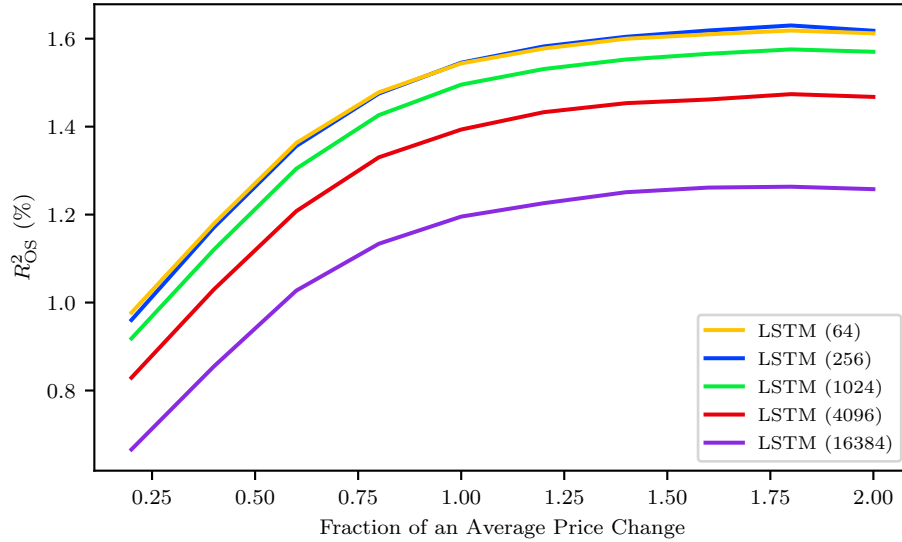


FIGURE 3. Out-of-sample forecasting performance of the LSTM model trained with batch sizes of 64, 256, 1024, 4096, 1638. Model performance is measured as the average R^2_{OS} across days and stocks. Horizons are given as the fraction of an average price change for each stock. Each model is trained and evaluated using a rolling-window out-of-sample methodology over thirteen months of data.

to the quite minor differences in network architecture or depth. Rather, we conjecture it is due to the nature of forecasts at varying time scales. Short-term signals are relatively easier to capture and do not require a large training set. However, for forecasts further out, more training data proves beneficial. In our study, each model is trained on three months of data, whereas in Kolm et al. (2023), the training sets consist of one month.

Since the performance of the LSTM, multi-head LSTM, and LSTM Seq2Seq models are similar, we will focus our analysis on the LSTM architecture in the following.

3.2. Batch Size. Investigating how batch size affects the out-of-sample performance of the LSTM, we train the model with batch sizes of 64, 256, 1024, 4096, and 16384, and present the results in Figure 3. While larger batch sizes generally offer more computationally efficient training by leveraging GPUs more effectively, our findings indicate that smaller batch sizes, such as 64 and 256, have a meaningful impact on forecasting performance. This observation is consistent with evidence from non-financial tasks, suggesting

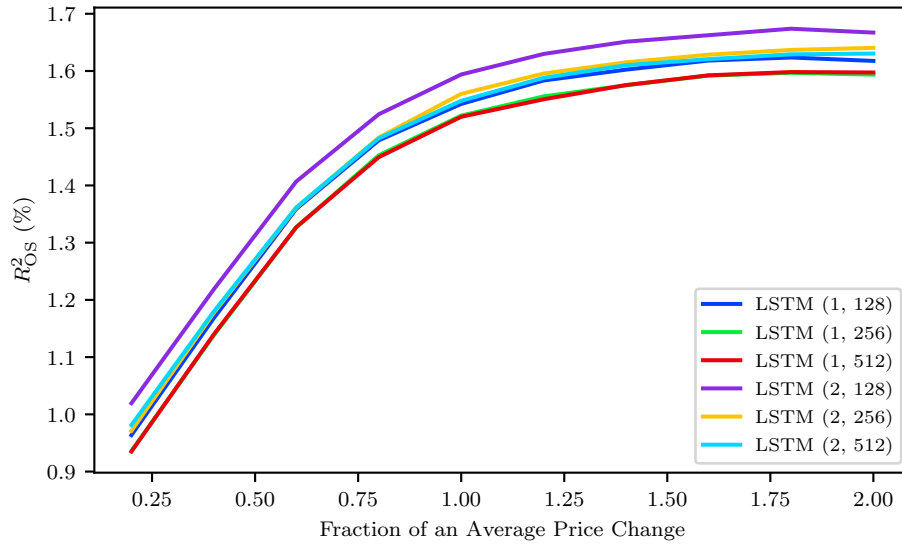


FIGURE 4. Out-of-sample forecasting performance of LSTM models of different depth (1 or 2) and width (128, 256 or 512). Model performance is measured as the average R^2_{OS} across days and stocks. Horizons are given as the fraction of an average price change for each stock. Each model is trained and evaluated using a rolling-window out-of-sample methodology over thirteen months of data.

that smaller batch sizes may aid in avoiding local minima and thus tend to generalize better (Keskar et al., 2017).

3.3. Width vs. Depth. The machine learning literature underscores the significance of *depth* (the number of hidden layers) as a fundamental component of modern neural networks. Increased depth enables models to learn more complex structures and typically offers greater explanatory power compared to that of increasing the *width* (the number of hidden units in the layers) (Goodfellow et al., 2016; He et al., 2016).

Comparing LSTM models with increased depth to those with increased width, we explore six configurations, varying the depth from 1 to 2, and the width across 128, 256 to 512. The results depicted in Figure 4 suggest a slight enhancement resulting from increasing the depth, albeit minimal. For a fixed depth, we observe a slight degradation in performance as width increases.

3.4. Time Features. Order flows are asynchronous and irregularly spaced time series that are updated whenever new information arrives in the order book. Therefore, time-based features related to the order flow updates, such as the time of day, the duration between updates, and seasonal patterns, could contain valuable information for forecasting mid-price returns. In this section, we study the performance of the LSTM model augmented with different time features.

We explore the following representations of time:

- *Duration between updates* (Δt): The time interval between consecutive order flow updates.
- *Time of day* (TOD): The time of day of order flow updates as a fraction of the trading day.
- *Time2Vec*: A 16-dimensional vector representation of the time of day of order flow updates (Kazemi et al., 2019).
- *Date features*: The day of the week, day of the month, and day of the year, of order flow updates represented as fractions, to allow the model to learn potential seasonal effects.

Introduced by Kazemi et al. (2019), Time2Vec is a technique for representing time in DNN models. It embeds time into a continuous vector space, enabling the capture of temporal patterns and dependencies in time-series data. Unlike traditional methods that rely on scalar representations of time (e.g., timestamps), Time2Vec learns a low-dimensional vector representation of time that can capture temporal relationships. In this article, we use a 16-dimensional representation.

Figure 5 illustrates improvement in average forecasting performance associated with time features such as TOD, Time2Vec (TOD), and Δt . Particularly, the combination of TOD and Δt shows the greatest improvement over the LSTM model without time features, denoted LSTM (No Time) in the figure. However, the LSTM model incorporating date features performs the worst, indicating the absence of significant seasonal effects.

In Figure 6, we depict the average forecasting performance of these models throughout the trading day, showing an inverse pattern compared to the well-known intraday volatility profile. Notably, incorporating TOD enhances performance, particularly towards the end of the day. The most significant improvement occurs by combining the Δt and TOD time features, consistent with Binkowski et al. (2018).

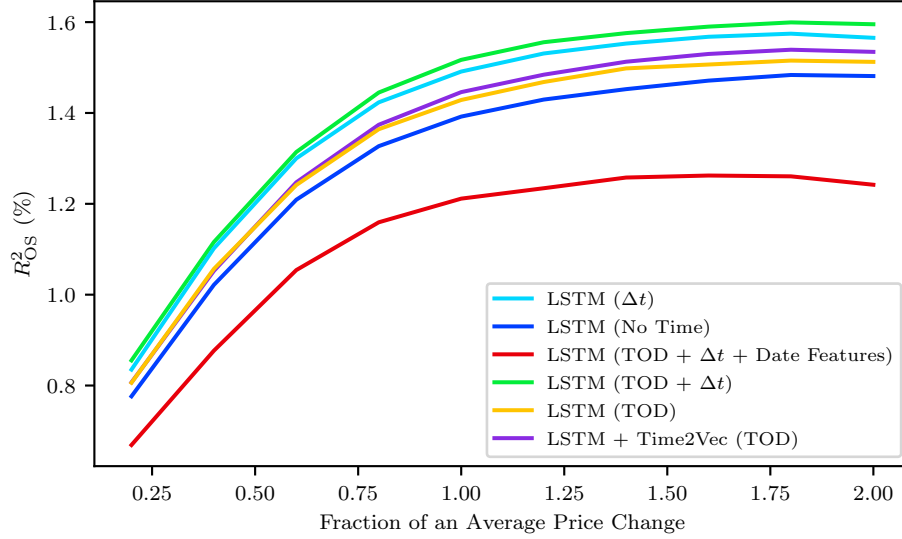


FIGURE 5. Out-of-sample forecasting performance of the LSTM model trained with different time features including duration between updates (Δt), time of day (TOD), a 16-dimensional Time2Vec representation of the time of day (Time2Vec (TOD)) and date features that allows the model to learn seasonal effects. LSTM (No Time) represents the LSTM model without time features. Model performance is measured as the average R^2_{OS} across days and stocks. Horizons are given as the fraction of an average price change for each stock. Each model is trained and evaluated using a rolling-window out-of-sample methodology over thirteen months of data.

3.5. Lookback Window Size. In the preceding sections, we maintained a lookback window size of $W = 100$ order book updates. Now, we investigate whether longer lookback windows enhance forecasting performance. It is worth mentioning that Kolm et al. (2023) show that for a similar LSTM model to the one we use here, expanding the size of the lookback window dynamically up to $W = 300$ does not yield improved performance compared to keeping $W = 100$ fixed. Furthermore, the average R^2_{OS} 's of LSTM models in Kolm et al. (2023, Fig. 8) reach their peak of about 1.3% around two average price changes, subsequently declining to 0.7% at ten average price changes. Nonetheless, it is conceivable that even longer histories may lead to different results. Therefore, we extend our analysis to horizons of up to 500 and 1000 order book updates. Instead of directly feeding the resulting OFIs to the LSTM, we condense them from $W \times 20$ to 100×20 using a linear convolutional layer, where W represents either 500 or 1000 OFIs. Thereafter,

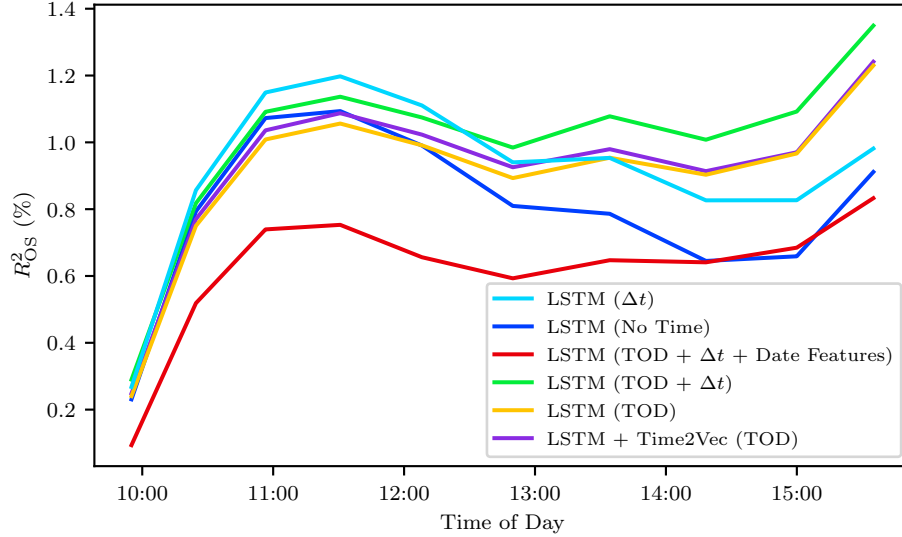


FIGURE 6. Out-of-sample forecasting performance throughout market hours of the LSTM model trained with different time features including duration between updates (Δt), time of day (TOD), a 16-dimensional Time2Vec representation of the time of day (Time2Vec(TOD)) and date features that allows the model to learn seasonal effects. LSTM(No Time) represents the LSTM model without time features. Model performance is measured as the average R^2_{OS} across intraday buckets, stocks, and horizons up to two average price changes. Each model is trained and evaluated using a rolling-window out-of-sample methodology over thirteen months of data.

we feed the output from the convolutional layer to the LSTM model. We denote these models by LSTM(500) and LSTM(1000) in the subsequent discussion.

Our results in Figure 7 show that the inclusion of longer term information does not enhance performance. Specifically, we observe that the average R^2_{OS} 's of the LSTM(500) and LSTM(1000) peak with values of around 1.3% and 1.2% at around two average price changes. Subsequently, the average R^2_{OS} 's decline to about 0.7% and 0.6% at ten average price changes, consistent with the performance of the LSTM models in Kolm et al. (2023, Fig. 8) with $W = 100$.

3.6. Order Book Stability Improves Forecasting Performance. Kolm et al. (2023) establish a connection between the forecasting accuracy of their LSTM model is and stock characteristics at the market microstructure level.

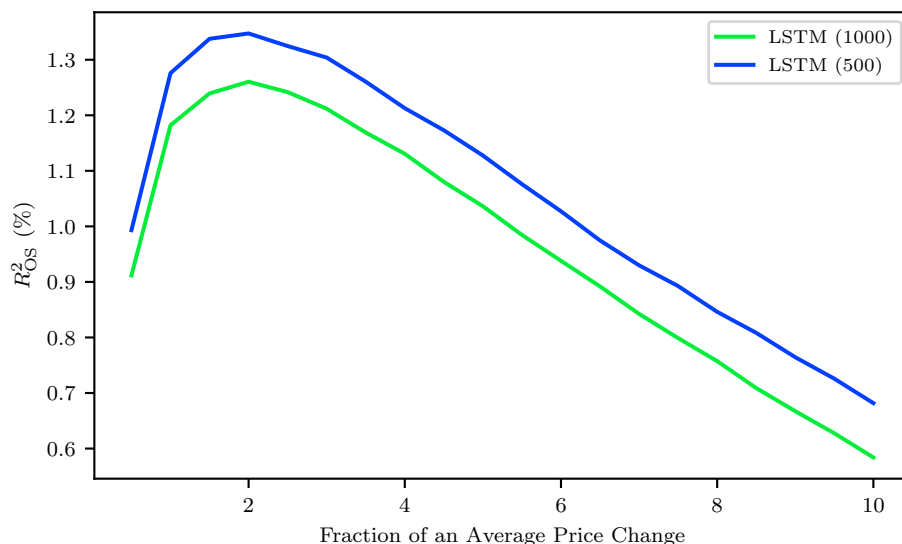


FIGURE 7. Out-of-sample forecasting performance the LSTM(500) and LSTM(1000) models. Each one of these models consists of a linear convolutional layer followed by an LSTM. Model performance is measured as the average R^2_{OS} across days and stocks. Horizons are given as the fraction of an average price change for each stock. Each model is trained and evaluated using a rolling-window out-of-sample methodology over thirteen months of data.

They demonstrate that “information-rich stocks,” which have a higher ratio of order book updates to price changes, exhibit more accurate forecasts. We provide further insights into the conditions under which the model may exhibit better performance.

Each time we use the models to forecast the alpha term structure of the stocks, we group the stocks into quintiles based on their cumulative first-level OFIs over the lookback window. This enables us to rank the stocks at each time point based on their cumulative first-level OFIs and divide them into quintiles. The first and last quintiles (Bin 0 and Bin 4) typically consist of stocks with large first-level OFIs of opposite sign. Subsequently, we calculate the R^2_{OS} across days and stocks for each quintile, depicting the results in the left panel of Figure 8. We observe that the lowest forecasting performance is associated with Bins 0 and 4. This observation aligns with the common understanding that significant order flow imbalances typically trigger order entry or cancellation, thereby altering the structure and depth

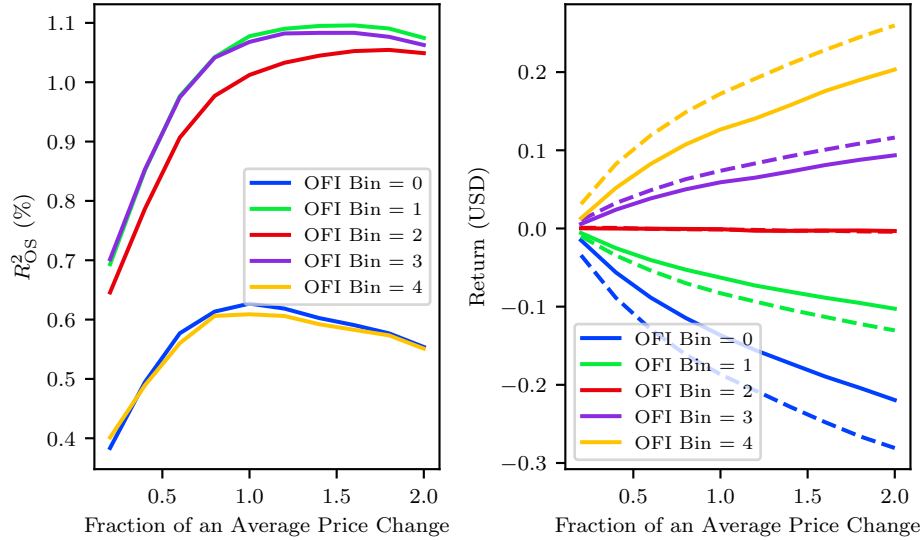


FIGURE 8. *Left panel:* Out-of-sample forecasting performance of the LSTM model for each OFI quintile. The quintiles are formed by ranking the stocks based on their cumulative first-level OFIs over the lookback window. Model performance is measured as the average R^2_{OS} across days and stocks. Horizons are given as the fraction of an average price change for each stock. Each model is trained and evaluated using a rolling-window out-of-sample methodology over thirteen months of data. *Right panel:* Average forecasted (solid line) and realized (dashed line) dollar returns for each OFI quintile.

of the order book. Consequently, such situations may lead to increased unpredictability of the market environment. Conversely, the models exhibit better performance when order flow imbalances are less pronounced, as seen in Bins 1, 2, and 3, where buying and selling interests are more balanced.

The right panel in Figure 8 illustrates the average forecasted (solid line) and realized (dashed line) dollar returns for the five bins. This highlights the effectiveness of the models in forecasting across the different market states defined by the bins, with slightly larger errors observed for Bins 0 and 4.

4. CONCLUSIONS

Recently, deep learning (DL) models have achieved significant success in forecasting high-frequency returns in equities by leveraging large limit order book data. In many of these studies, features (predictors) are learned directly from the data rather than being manually crafted, representing a significant departure from current industry practice. While there is some convergence in

consensus in the literature, several practical open questions remain regarding network architecture and input selection. In this article, we aimed to address a number of these.

Initially, we evaluated the effectiveness of four DL models in forecasting high-frequency alpha term structures across various settings: (i) a simple LSTM, (ii) a multi-head LSTM, (iii) an LSTM Seq2Seq without attention, and (iv) an LSTM Seq2Seq with attention. Our empirical findings revealed that surpassing the performance of a simple LSTM in the return forecasting task is surprisingly challenging. Specifically, we observed comparable performance among the LSTM, multi-head LSTM, and LSTM Seq2Seq models, while the attention-based LSTM Seq2Seq model demonstrated inferior performance compared to the others. Therefore, with our focus on the LSTM for the rest of our empirical work, we presented the following findings:

- Batch size: Although larger batch sizes result in more computationally efficient training by leveraging GPUs more effectively, smaller batch sizes, such as 64 and 256, have a meaningful impact on forecasting performance.
- Width vs. depth: Varying depth and width, we showed there is a slight improvement with increased depth, albeit minimal. Keeping depth fixed, we observed a slight degradation in performance as width increases.
- Time features: Analyzing improvement in forecasting performance associated with different time features, we found that the combination of time of day, and the duration between LOB updates, shows the greatest improvement. Conversely, the LSTM model incorporating date features performed the worst, indicating the absence of significant seasonal effects.
- Lookback window size: Including longer-term information beyond a few hundred order book updates did not improve performance.
- Finally, we provided additional support to the findings of Kolm et al. (2023) that a more stable order book leads to stronger forecasting performance.

ACKNOWLEDGEMENTS

This work was supported in part through the NYU IT High Performance Computing resources, services, and staff expertise. In particular, we are grateful to Shenglong Wang for his assistance when developing our data processing and computing environment on NYU’s Greene and Hudson high-performance computing clusters.

APPENDIX A. DEEP LEARNING FOR ALPHA FORECASTING

A.1. Architectures.

A.1.1. *Long Short-Term Memory.* The key elements of the LSTM are (i) a memory cell, representing its state over time, and (ii) non-linear gates that determine the information flow in and out of the memory cell. It is common to use three gates known as *input*, *output*, and *forget gates* (Gers et al., 2000). The LSTM unit is characterized by the following equations, with $\mathbf{x}_t \in \mathbb{R}^N$ representing the input vector⁶

$$\mathbf{f}_t = \sigma(\mathbf{U}^f \mathbf{x}_t + \mathbf{W}^f \mathbf{h}_{t-1} + \mathbf{b}^f) \quad (8)$$

$$\mathbf{i}_t = \sigma(\mathbf{U}^i \mathbf{x}_t + \mathbf{W}^i \mathbf{h}_{t-1} + \mathbf{b}^i) \quad (9)$$

$$\mathbf{o}_t = \sigma(\mathbf{U}^o \mathbf{x}_t + \mathbf{W}^o \mathbf{h}_{t-1} + \mathbf{b}^o) \quad (10)$$

$$\mathbf{s}_t = \mathbf{f}_t \circ \mathbf{s}_{t-1} + \mathbf{i}_t \circ \tanh(\mathbf{U}^s \mathbf{x}_t + \mathbf{W}^s \mathbf{h}_{t-1} + \mathbf{b}^s) \quad (11)$$

$$\mathbf{h}_t = \mathbf{o}_t \circ \tanh(\mathbf{s}_t), \quad (12)$$

where $\sigma := (1 + e^{-x})^{-1}$ is the sigmoid activation function, $\mathbf{f}_t \in \mathbb{R}^M$ is the forget gate’s activation vector, $\mathbf{i}_t \in \mathbb{R}^M$ is the input gate’s activation vector, $\mathbf{o}_t \in \mathbb{R}^M$ is the output gate’s activation vector, $\mathbf{h}_t \in \mathbb{R}^M$ is the output vector of the LSTM unit, $\mathbf{s}_t \in \mathbb{R}^M$ is the unit’s hidden state vector, and $\mathbf{W} \in \mathbb{R}^{M \times N}$, $\mathbf{U} \in \mathbb{R}^{M \times M}$ and $\mathbf{b} \in \mathbb{R}^M$ (superscripts dropped) are weight matrices and bias vector parameters that are learned during training. Here, $\circ$ represents the element-wise product and $\tanh$ denotes the hyperbolic tangent function. To mitigate the vanishing gradient problem during training, we initialize $\mathbf{b}^f$ to one, following the approach in (Gers et al., 2000). For our regression-based supervised learning problem we feed the output from the LSTM unit through a dense layer with a nonlinear activation function.

⁶This formulation is implemented in Keras, accessible at https://www.tensorflow.org/api_docs/python/tf/keras/layers/LSTM. We employ this implementation in our empirical analysis.

The multi-head LSTM we employ in our empirical work uses the same architecture as the LSTM above, with each head having its own LSTM unit and parameter set.

A.1.2. *Sequence to Sequence.* Our implementation of the Seq2Seq models is built upon the code by Zhang and Zohren (2021), which draws from the techniques outlined in the works of Cho et al. (2014) and Luong et al. (2015).

A Seq2Seq model consists of (i) an encoder, and (ii) a decoder. The encoder is a *recurrent neural network* (RNN) that sequentially processes an input sequence $\{\mathbf{x}_t\}_{t=1}^T$, where $\mathbf{x}_t \in \mathbb{R}^N$ for all t . At each time step t , the hidden state of the RNN, $\mathbf{h}_t^e \in \mathbb{R}^{M_e}$, is updated according to

$$\mathbf{h}_t^e = f_e(\mathbf{h}_{t-1}^e, \mathbf{x}_t), \quad (13)$$

where f_e is a nonlinear activation function. After processing the final input in the sequence, the encoder outputs the final hidden state, $\mathbf{c} := \mathbf{h}_T^e$, referred to as the *context vector*, providing a summary of the entire input sequence. The decoder, also an RNN, updates its hidden state, $\mathbf{h}_t^d \in \mathbb{R}^{M_d}$, at each time step t , via

$$\mathbf{h}_t^d = f_d(\mathbf{h}_{t-1}^d, \mathbf{y}_{t-1}, \mathbf{c}), \quad (14)$$

and generates the output, $\mathbf{y}_t \in \mathbb{R}^H$, by

$$\mathbf{y}_t = g(\mathbf{h}_t^d, \mathbf{c}), \quad (15)$$

where f_d and g are nonlinear activation functions. For the regression task in this article, we use a dense layer for g .

Seq2Seq models are known to face challenges when dealing with longer sequences. A reason for this is the difficulty in capturing long-term dependencies and summarizing the entire input sequence into a fixed-size context vector. Specifically, the model may struggle to retain important information from earlier parts of the sequence, leading to performance degradation with increasing sequence length.

The Seq2Seq + Attn model by Luong et al. (2015) is designed to effectively handle longer sequences and capture more relevant information from the input sequence during decoding. The attention mechanism allows the decoder to selectively focus on different parts of the input sequence while generating the output sequence. Rather than using a fixed context vector, the decoder dynamically attends to different segments of the input sequence during each decoding step based on their relevance using the dynamic context vector $\mathbf{c}_t$,

given by

$$\mathbf{c}_t := \sum_{s=1}^T \alpha_{t,s} \mathbf{h}_s^e, \quad (16)$$

where the *attention weights*, $\alpha_{t,s} \geq 0$, are given by

$$\alpha_{t,s} := \frac{\exp(\text{score}(\mathbf{h}_{t-1}^d, \mathbf{h}_s^e))}{\sum_{\tau=1}^T \exp(\text{score}(\mathbf{h}_{t-1}^d, \mathbf{h}_\tau^e))}, \quad (17)$$

with the content-based *score* function defined by one of the following three alternatives (Luong et al., 2015)

$$\text{score}(\mathbf{h}_{t-1}^d, \mathbf{h}_s^e) := \begin{cases} (\mathbf{h}_s^e)' \mathbf{h}_{t-1}^d & (\text{dot}), \\ (\mathbf{h}_s^e)' \mathbf{W}_a \mathbf{h}_{t-1}^d & (\text{general}), \\ (\mathbf{v}_a)' \tanh(\mathbf{W}_a [\mathbf{h}_s^e; \mathbf{h}_{t-1}^d]) & (\text{concatenate}), \end{cases} \quad (18)$$

where $\mathbf{v}_a \in \mathbb{R}^{M_a}$ and $\mathbf{W}_a \in \mathbb{R}^{M_a \times (M_e + M_d)}$ are a weight vector and matrix, respectively, learned during training. Then, at each time t , we update the decoder and generate the output, $\mathbf{y}_t$, via

$$\mathbf{h}_t^d = f_d(\mathbf{h}_{t-1}^d, \mathbf{y}_{t-1}, \mathbf{c}_t), \quad (19)$$

$$\mathbf{y}_t = g(\mathbf{h}_t^d, \mathbf{c}_t), \quad (20)$$

where f_d and g are nonlinear activation functions, as above.

A.2. Data. We source our data from LOBSTER (Huang and Polak, 2011), which offers complete order book state information from Nasdaq, excluding submissions and cancellations of hidden orders. We select all stocks that meet the following criteria from January 1, 2019, through January 31, 2020: (i) membership in the Nasdaq 100 index at some point during the period, (ii) availability of data for at least 260 trading days, and (iii) absence of corporate actions during the study period. We exclude January 9, 2019, due to data issues. This yields a universe of 115 stocks, including well-known names such as Amazon (AMZN), American Airlines (AAL), Facebook (FB), Google (GOOGL), Microsoft (MSFT), and Netflix (NFLX). The larger number of stocks and longer time period compared to most previous studies provide greater confidence in our results.

We preprocess the LOBSTER data to address scenarios such as a single market order executing against multiple limit orders, or a modification to a limit order being processed as a cancellation followed by a new submission

(Bouchaud et al., 2018; Bugaenko, 2020). We describe our preprocessing steps in Appendix B.

Our preprocessed dataset, detailed in Table 2 in Appendix B, mirrors the dataset used in Kolm et al. (2023) and Kolm and Westray (2023), totaling about 10TB uncompressed. Consequently, storage and efficient processing pose significant challenges. Our dataset is one to two orders of magnitude larger than those in similar studies applying DNNs to limit order books.⁷ The datasets used by Sirignano (2019) and Sirignano and Cont (2019) are exceptions. Their universe, drawn from the S&P 500, is larger than ours. However, the focus of their work differs from ours in that they train classifiers on downsampled time series of order book updates, and Sirignano and Cont (2019) investigate whether the function $\mathbf{g}$ in (7) is universal. It is well-known that not only trades but all changes to the order book are vital in understanding price formation (Cont et al., 2014; Bouchaud et al., 2018). Therefore, as our focus in this article is forecasting alpha term structures rather than questions of universality, and downsampling may result in removing potentially important information, we use all order book updates and train DNNs for each symbol in the dataset.

A.2.1. Construction of Dependent and Independent Variables. Our stock universe comprises a diverse range of symbols with significantly varying order book activity. For example, EBAY and MSFT exhibit three orders of magnitude difference in terms of the number of order book updates per day. These disparate update frequencies are associated with the concept known as *stock characteristic time*, often formalized by intermittent processes such as Hawkes processes (Jaisson and Rosenbaum, 2016; Bouchaud et al., 2018). To tailor the horizons of the alpha term structure in (6) to each stock, following Kolm et al. (2023), we use the stock-specific time increment

$$\Delta t := \frac{2.34 \cdot 10^7}{N}, \quad (21)$$

where the numerator represents the number of milliseconds in a trading day, while the denominator, denoted by N , signifies the stock's average number of non-zero tick-by-tick mid-price returns. This ratio reflects the average waiting time for a change in the mid-price of a specific stock.

⁷Tsantekidis et al. (2017), Tran et al. (2019), Zhang et al. (2018), Zhang et al. (2019a), Zhang et al. (2019b), Tsantekidis et al. (2020), Briola et al. (2020), Zhang et al. (2021), and Zhang and Zohren (2021).

Unless explicitly noted otherwise, we select an alpha term structure (6) of dimension $H = 10$, and define our forecast horizons, $\{h_k\}_{k=1}^{10}$, as the multiples of Δt , in particular

$$h_k := \frac{1}{5}k\Delta t, k = 1, \dots, 10. \quad (22)$$

We argue in Kolm et al. (2023) that this choice of stock-specific forecast horizons adjust the time scale appropriately for each stock. While there is potential to improve the specification of these horizons to consider various intraday effects, this approach allows for sensible comparison of results across stocks. For instance, incorporating intraday profiles is possible within this framework (see, for example, Mertens et al., 2020).

To account for time delays associated with computing model forecasts and round-trip communication between exchanges and trading algorithms, it is standard practice to incorporate latency buffers during alpha development and testing. According to Hasbrouck and Saar (2013), using Nasdaq-sourced data for one month in 2007 and 2008, respectively, the fastest traders in their sample have an effective latency of 2–3 milliseconds. Additionally, Zhang et al. (2019a) report that the average time required to produce a return forecast from various state-of-the-art DNNs range from 0.03 to 0.97 milliseconds. To err on the conservative side, we adopt a latency buffer of ten milliseconds in this article.

In our single stock forecasting models (7), the h_k -horizon mid-price returns $r_{t,k} := p_{t+h_k} - p_{t+\delta\tau}$ (in dollars) serve as dependent variables, where p_t denotes the mid-price at time t , and $\delta\tau$ represents the ten-millisecond latency buffer. We assess model performance by computing out-of-sample returns using the same methodology.

While Kolm et al. (2023) consider full order book and OF inputs, in this article we use ten-level OFIs as independent variables, which are reduced representations of order book states, known to be stationary (Cont et al., 2014).

In our training sets, we apply winsorization to all dependent and independent variables at the 0.5% and 99.5% levels, followed by normalization of the resulting data using Z -scores. We retain the normalization parameters obtained from the in-sample data to scale the test and validation sets for out-of-sample evaluation.

A.3. Training and Evaluation. Our training and evaluation involves a quarterly schedule over thirteen months, using a rolling-window with a (1 month, 3 months, 3 months)-structure. The first month is used for validation, the subsequent three months are dedicated to training, and the last three months are reserved for out-of-sample testing. During model training, we minimize the *mean-squared error* (MSE) using *stochastic gradient descent* (SGD) with the Adam optimizer (Kingma and Ba, 2014). Unless explicitly noted otherwise, we use a relatively small batch size of 256, as evidence suggests that smaller batch sizes lead to local minima that generalize better (Keskar et al., 2017).

When concatenating the time series of order book updates and returns, it is important to not mix data from different trading days within the same batch. To address this issue, we develop a custom Keras generator. To prevent overfitting, we use early stopping for all models, terminating training when the validation loss remains unchanged for five consecutive epochs. We use Python to code our models, leveraging Tensorflow (Abadi et al., 2015) and Keras (Chollet et al., 2015) for implementing the DNNs.

We evaluate the performance of model forecasts at the individual stock level using *out-of-sample R^2 at horizon h* ($R_{OS,h}^2$) for each test period, defined as

$$R_{OS,h}^2 := 1 - \text{MSE}_{m,h} / \text{MSE}_{\text{bmk},h}, \quad (23)$$

where $\text{MSE}_{m,h}$ and $\text{MSE}_{\text{bmk},h}$ are the mean-squared errors of the model forecasts and benchmark at the h -th horizon, respectively. As a conservative approach, we employ the average out-of-sample return as our benchmark. If $R_{OS,h}^2 > 0$, it indicates that the model outperforms the benchmark. Each forecast relies solely on data available up to the time of its computation. When the time horizon is evident, we omit the subscript h and denote it as R_{OS}^2 .

We train our models on NYU’s Greene⁸ and Hudson⁹ high performance computing environments. This infrastructure allows us to massively parallelize the data processing and training of the models. The time to train a single model varies in the range of 10-60 minutes, depending on model and stock.

⁸Greene has 32K CPU cores, 332 NVIDIA GPUs and 145TB of RAM distributed over 568 nodes.

See, <https://www.nyu.edu/about/news-publications/news/2020/november/GreeneSupercomputer.html>.

⁹Hudson has 960 CPU cores, 160 AMD GPUs and 10TB of RAM distributed across 20 nodes. See, <https://wp.nyu.edu/connect/2020/06/08/nyu-expands-supercomputing-amd>.

APPENDIX B. PREPROCESSING OF THE LOBSTER DATA

In this section we discuss the format of the LOBSTER data and summarize the preprocessing steps we performed for this article. For additional information we refer to Bouchaud et al. (2018, Appendix A) and Bugaenko (2020, Appendix C).

LOBSTER provides two files for each date and symbol, the message and order book files. The number of rows in each file is identical and the timestamps are given to nanosecond precision. The message file lists every market order arrival, limit order arrival and cancellation. We note that the arrival of hidden orders is not provided. The order book file lists the states of the limit order book with each row corresponding to the state after the corresponding event from the message file. It is the order book file which is the primary object of study in this paper.

Inspection of the message file reveals that there are certain clusters of events that require special care. In particular, consider the following three situations:

- (i) Limit order modification which is implemented as a cancellation followed by an immediate new arrival.
- (ii) Single market order executing against multiple resting limit orders.
- (iii) Auction prints: The auction trade is printed and all the remaining resting limit orders are moved into the limit order book.

Common amongst these situations is that we have one conceptual event which appears as multiple rows in the message file, all with the same timestamp. Consequently, it is critical for modeling purposes that these events are collapsed to avoid artificially inflated prediction accuracy. Therefore, we group the order book data by unique timestamps and take the last entry (they are ordered sequentially within a timestamp by the process which generates this data). This type of processing is standard in the literature using LOBSTER data (see, for example, Bouchaud et al. (2018) and Bugaenko (2020)). In addition, we remove all rows for which the quotes are crossed. As noted in Bouchaud et al. (2018) the occurrence of such events is extremely rare.

Table 2 provides summary statistics after preprocessing the LOBSTER data for the 115 stocks used in this study.

Ticker	Updates (000)	Trades (000)	Price Changes (000)	Price (USD)	Spread (bps)	Volume (USD MM)
AAL	398.13	13.15	10.99	30.65	3.88	50.29
AAPL	1137.36	64.19	127.86	215.80	0.99	1156.30
ADBE	205.29	13.20	41.76	283.83	4.08	194.36
ADI	238.35	10.70	29.23	109.36	3.52	73.59
ADP	128.11	7.82	20.02	160.65	3.62	72.97
ADSK	118.22	8.34	24.27	161.21	5.48	74.96
ALGN	80.86	6.78	20.01	251.14	9.68	81.68
ALXN	93.19	7.47	18.08	117.26	7.45	52.07
AMAT	611.97	18.05	17.15	47.36	2.62	91.72
AMD	1213.04	42.06	16.26	31.33	3.64	343.63
AMGN	137.56	11.06	25.61	198.36	3.85	138.33
AMZN	304.76	31.14	64.23	1795.51	2.59	1531.40
ANSS	63.55	3.37	15.22	206.76	9.39	25.86
ASML	147.63	3.31	21.21	223.92	7.06	46.42
ATVI	423.01	17.97	20.25	49.87	2.81	91.82
AVGO	136.68	12.20	32.10	289.33	4.62	182.39
BIDU	155.90	13.37	26.50	132.58	4.79	129.79
BIIB	88.20	9.80	23.17	265.36	6.01	129.65
BKNG	51.44	4.20	15.00	1881.52	8.68	169.08
BMRN	61.69	5.08	10.81	82.87	10.20	25.70
CDNS	128.56	7.32	15.40	64.99	4.28	32.49
CDW	53.73	4.20	9.73	112.53	7.93	28.02
CERN	117.85	7.57	10.59	66.41	3.53	38.58
CHKP	67.19	3.98	10.82	114.42	6.51	27.00
CHTR	80.97	6.75	19.21	400.68	6.22	111.71
CMCSA	612.78	18.70	7.44	42.40	2.58	128.95
COST	125.41	9.21	26.87	264.93	3.26	120.40
CPRT	80.99	4.77	9.66	73.53	5.03	23.75
CSCO	736.33	23.41	9.81	50.37	2.17	191.94
CSGP	38.27	2.08	9.03	536.05	23.06	34.75
CSX	294.56	12.52	18.20	72.07	2.35	78.23
CTAS	64.05	3.71	14.75	237.20	7.29	34.05
CTSH	271.85	10.22	14.73	64.84	2.39	58.60
CTXS	103.49	7.60	12.32	102.47	3.55	52.46
DISH	138.93	7.27	9.16	34.30	4.95	21.35
DLTR	132.57	8.14	15.42	102.30	4.23	57.93
EA	198.67	13.38	26.25	96.82	4.19	95.90
EBAY	435.54	12.59	5.91	37.05	2.98	66.41
EXC	298.32	7.58	7.21	47.35	2.45	37.55
EXPE	122.44	8.33	16.93	122.62	4.45	62.15
FAST	229.64	8.64	8.99	44.16	3.47	34.60
FB	602.54	44.01	106.38	184.08	1.71	622.29
FISV	178.66	12.38	20.43	97.41	3.14	90.26
FOX	199.41	4.56	9.31	37.81	4.33	19.34
FOXA	239.37	9.02	8.03	38.22	3.39	44.54
GILD	407.04	15.22	20.67	65.53	2.22	96.45
GLIBA	47.21	2.06	6.55	61.12	15.97	7.51
GOOG	590.59	13.38	64.96	1206.29	4.71	468.31
GOOGL	517.61	14.61	62.70	1208.90	4.40	445.79
HAS	73.34	5.16	11.42	101.28	6.61	30.45
HOLX	134.48	5.45	7.55	48.02	3.85	20.87
HSIC	99.05	5.18	9.01	66.23	5.41	23.73
IDXX	67.22	3.39	16.85	251.26	9.72	30.08
ILMN	79.51	6.50	20.37	310.12	7.77	84.54
INCY	72.65	5.39	12.91	81.55	8.74	26.16

Continued on next page

Ticker	Updates (000)	Trades (000)	Price Changes (000)	Price (USD)	Spread (bps)	Volume (USD MM)
INTC	966.81	29.58	12.86	52.13	2.10	231.35
INTU	100.24	7.37	23.84	257.26	5.30	88.90
ISRG	78.95	5.84	20.09	535.71	7.83	94.48
JBHT	73.90	4.76	12.57	104.88	7.74	28.07
JD	552.74	18.58	10.95	29.94	3.79	95.11
KHC	351.72	12.51	6.75	32.58	3.48	58.06
KLAC	94.44	7.33	19.12	136.17	6.19	53.08
LBTYA	217.71	5.40	6.73	24.95	5.39	13.11
LBTYK	262.49	6.83	5.15	24.16	4.99	19.60
LILAK	49.89	1.81	3.77	18.09	15.92	2.63
LRCX	154.50	10.43	30.42	215.34	5.48	111.39
LULU	92.51	8.25	19.44	184.49	6.34	87.43
MAR	116.61	6.94	17.30	130.38	4.51	54.67
MCHP	146.21	8.99	19.50	91.11	4.95	51.90
MDLZ	385.19	11.27	6.57	51.78	2.16	68.01
MELI	68.15	3.28	16.99	538.46	19.44	72.08
MNST	207.65	9.02	12.41	59.63	3.14	45.60
MSFT	1314.51	50.80	59.44	132.70	0.96	674.33
MU	1038.48	33.55	23.06	43.55	2.64	224.59
MXIM	165.32	6.96	13.37	57.14	4.29	28.77
MYL	280.72	10.56	6.72	22.25	5.33	30.69
NFLX	299.09	29.56	69.40	329.25	3.92	554.70
NTAP	186.58	9.14	16.83	60.89	4.14	40.97
NTES	59.40	4.19	14.83	268.10	13.15	55.21
NVDA	410.14	32.36	78.34	179.73	3.12	388.83
NXPI	156.79	11.93	21.49	103.13	5.01	88.60
ORLY	71.82	5.06	18.57	393.07	7.90	69.74
PAYX	204.00	7.03	15.93	81.79	2.86	38.72
PCAR	131.38	6.85	13.03	70.63	3.82	34.43
PEP	307.10	13.89	26.13	128.67	1.73	125.57
PYPL	381.85	19.86	39.32	105.65	2.21	164.14
QCOM	547.61	27.86	31.30	73.04	2.23	213.91
QRTEA	121.32	5.23	2.79	13.54	9.75	9.53
REGN	72.53	5.74	18.86	344.93	9.32	70.71
ROST	155.81	8.53	17.27	103.03	3.85	55.34
SBUX	485.99	17.70	19.99	82.04	1.63	139.81
SGEN	44.67	3.95	9.96	82.34	15.15	19.56
SIRI	107.42	4.50	0.29	6.21	16.69	25.20
SNPS	85.39	5.32	15.31	125.01	5.77	35.22
SPLK	84.00	7.54	19.09	129.00	8.36	57.97
STX	226.02	9.21	14.74	50.18	3.67	38.19
SWKS	114.57	8.55	16.98	86.36	5.71	46.25
TMUS	194.31	9.84	15.56	75.70	3.08	60.40
TSLA	283.15	38.43	65.03	292.35	4.66	724.21
TTWO	101.33	7.31	18.13	112.77	6.62	51.14
TXN	324.28	16.38	39.16	116.63	2.43	136.22
UAL	163.59	9.65	17.19	86.65	3.98	59.90
ULTA	77.25	6.45	19.19	297.27	8.64	79.56
VOD	195.93	2.98	0.74	18.56	5.62	14.53
VRSK	71.10	3.96	12.71	142.97	5.62	28.14
VRSN	69.03	4.10	15.10	191.87	6.80	34.27
VRTX	76.14	6.16	17.19	187.81	7.44	58.90
WBA	328.50	13.20	15.18	57.81	2.54	75.19
WDAY	87.49	8.63	20.57	185.12	8.02	86.66
WDC	343.08	17.03	24.69	51.91	3.89	77.10
WLTW	50.77	3.47	11.74	185.22	8.16	27.11

Continued on next page

Ticker	Updates (000)	Trades (000)	Price Changes (000)	Price (USD)	Spread (bps)	Volume (USD MM)
WYNN	90.56	8.39	19.15	123.37	7.61	64.33
XEL	325.18	8.60	10.32	59.35	2.06	44.25
XLNX	200.22	14.14	34.38	107.30	4.31	103.57
XRAY	83.07	5.41	7.11	52.32	4.35	21.67

TABLE 2. Summary statistics of the 115 stocks from Nasdaq used in this study. The data is sourced from LOBSTER, covering the time period January 1, 2019 through January 31, 2020. Updates, trades, and price changes represent the average number of order book updates, trades and price changes in thousands per day. Price, spread, and volume denote the average price in dollars, bid-ask spread in basis points, and daily volume in dollars, respectively.

REFERENCES

- Abadi, Martín et al. (2015). *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. Software available from tensorflow.org. URL: <https://www.tensorflow.org/>.
- Abergel, Frédéric, Marouane Anane, Anirban Chakraborti, Aymen Jedidi, and Ioane Muni Toke (2016). *Limit Order Books*. Cambridge University Press.
- Achiam, Josh, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. (2023). “GPT-4 Technical Report”. In: *arXiv preprint arXiv:2303.08774*.
- Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio (2014). “Neural Machine Translation by Jointly Learning to Align and Translate”. In: *arXiv preprint arXiv:1409.0473*.
- Bilokon, Paul and Yitao Qiu (2023). “Transformers Versus LSTMs for Electronic Trading”. In: *arXiv preprint arXiv:2309.11400*.
- Binkowski, Mikolaj, Gautier Marti, and Philippe Donnat (2018). “Autoregressive Convolutional Neural Networks for Asynchronous Time Series”. In: *International Conference on Machine Learning*. PMLR, pp. 580–589.
- Bouchaud, Jean-Philippe, Julius Bonart, Jonathan Donier, and Martin Gould (2018). *Trades, Quotes and Prices: Financial Markets Under the Microscope*. Cambridge University Press.
- Briola, Antonio, Silvia Bartolucci, and Tomaso Aste (2024). “Deep Limit Order Book Forecasting”. In: *arXiv preprint arXiv:2403.09267*.

- Briola, Antonio, Jeremy Turiel, and Tomaso Aste (2020). *Deep Learning Modeling Of Limit Order Book: A Comparative Perspective*. arXiv: 2007.07319 [q-fin.TR].
- Brown, Tom, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. (2020). “Language Models Are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems* 33, pp. 1877–1901.
- Bugaenko, Anastasia (2020). *Empirical Study of Market Impact Conditional on Order-Flow Imbalance*. arXiv: 2004.08290 [q-fin.CP].
- Cho, Kyunghyun, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio (2014). “Learning Phrase Representations Using RNN Encoder-Decoder For Statistical Machine Translation”. In: *arXiv preprint arXiv:1406.1078*.
- Chollet, François et al. (2015). *Keras*. <https://github.com/fchollet/keras>.
- Chorowski, Jan K., Dzmitry Bahdanau, Dmitriy Serdyuk, Kyunghyun Cho, and Yoshua Bengio (2015). “Attention-Based Models for Speech Recognition”. In: *Advances in Neural Information Processing Systems* 28.
- Cont, Rama, Mihai Cucuringu, and Chao Zhang (2023). “Cross-Impact of Order Flow Imbalance in Equity Markets”. In: *Quantitative Finance*. DOI: DOI:10.1080/14697688.2023.2236159.
- Cont, Rama, Arseniy Kukanov, and Sasha Stoikov (2014). “The Price Impact of Order Book Events”. In: *Journal of Financial Econometrics* 12.1, pp. 47–88.
- Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova (2018). “BERT: Pre-Training Of Deep Bidirectional Transformers For Language Understanding”. In: *arXiv preprint arXiv:1810.04805*.
- Fan, Chenyou, Yuze Zhang, Yi Pan, Xiaoyue Li, Chi Zhang, Rong Yuan, Di Wu, Wensheng Wang, Jian Pei, and Heng Huang (2019). “Multi-Horizon Time Series Forecasting with Temporal Attention Learning”. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 2527–2535.
- Gers, Felix A., Jürgen Schmidhuber, and Fred Cummins (2000). “Learning to Forget: Continual Prediction with LSTM”. In: *Neural Computation* 12.10, pp. 2451–2471.

- Goodfellow, Ian, Yoshua Bengio, and Aaron Courville (2016). *Deep Learning*. MIT press.
- Gould, Martin D., Mason A. Porter, Stacy Williams, Mark McDonald, Daniel J. Fenn, and Sam D. Howison (2013). “Limit Order Books”. In: *Quantitative Finance* 13.11, pp. 1709–1742.
- Hasbrouck, Joel and Gideon Saar (2013). “Low-Latency Trading”. In: *Journal of Financial Markets* 16.4, pp. 646–679.
- He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun (2016). “Deep Residual Learning for Image Recognition”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778.
- Hochreiter, Sepp and Jürgen Schmidhuber (1997). “Long Short-Term Memory”. In: *Neural Computation* 9.8, pp. 1735–1780.
- Huang, Ruihong and Tomas Polak (2011). “LOBSTER: Limit Order Book Reconstruction System”. In: *Available at SSRN 1977207*.
- Jaisson, Thibault and Mathieu Rosenbaum (2016). “Rough Fractional Diffusions As Scaling Limits Of Nearly Unstable Heavy Tailed Hawkes Processes”. In: *The Annals of Applied Probability* 26.5, pp. 2860–2882.
- Kazemi, Seyed Mehran, Rishab Goel, Sepehr Eghbali, Janahan Ramanan, Jaspreet Sahota, Sanjay Thakur, Stella Wu, Cathal Smyth, Pascal Poupard, and Marcus Brubaker (2019). “Time2Vec: Learning a Vector Representation of Time”. In: *arXiv preprint arXiv:1907.05321*.
- Keskar, Nitish Shirish, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang (2017). *On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima*. arXiv: 1609 . 04836 [cs.LG].
- Kingma, Diederik P. and Jimmy Ba (2014). *Adam: A Method for Stochastic Optimization*. arXiv: 1412.6980 [cs.LG].
- Kolm, Petter N., Jeremy Turiel, and Nicholas Westray (2023). “Deep Order Flow Imbalance: Extracting Alpha at Multiple Horizons from the Limit Order Book”. In: *Mathematical Finance* 33.4, pp. 1044–1081.
- Kolm, Petter N and Nicholas Westray (2023). “Information Content of Cross-Sectional and Multilevel Order Flow Imbalances: A Bayesian Approach”. In: *Available at SSRN 4568641*.
- LeCun, Yann and Yoshua Bengio (1995). “Convolutional Networks for Images, Speech, and Time Series”. In: *The Handbook of Brain Theory and Neural Networks* 3361.10, p. 1995.

- LeCun, Yann, Léon Bottou, Yoshua Bengio, and Patrick Haffner (1998). “Gradient-Based Learning Applied to Document Recognition”. In: *Proceedings of the IEEE* 86.11, pp. 2278–2324.
- Lucchese, Lorenzo, Mikko Pakkanen, and Almut Veraart (2022). “The Short-Term Predictability of Returns in Order Book Markets: A Deep Learning Perspective”. In: *arXiv preprint arXiv:2211.13777*.
- Luo, Wei and Feng Yu (2019). “Recurrent Highway Networks With Grouped Auxiliary Memory”. In: *IEEE Access* 7, pp. 182037–182049.
- Luong, Minh-Thang, Hieu Pham, and Christopher D. Manning (2015). “Effective Approaches to Attention-Based Neural Machine Translation”. In: *arXiv preprint arXiv:1508.04025*.
- Mäkinen, Ymir, Juho Kanninen, Moncef Gabbouj, and Alexandros Iosifidis (2019). “Forecasting Jump Arrivals In Stock Prices: New Attention-Based Network Architecture Using Limit Order Book Data”. In: *Quantitative Finance* 19.12, pp. 2033–2050.
- Mertens, Luca, Alberto Ciacchi, Fabrizio Lillo, and Giulia Livieri (2020). “Liquidity Fluctuations And The Latent Dynamics Of Price Impact”. In: *Available at SSRN 3214744*.
- Newborn, Monty (2012). *Kasparov versus Deep Blue: Computer Chess Comes of Age*. Springer Science & Business Media.
- Prata, Matteo, Giuseppe Masi, Leonardo Berti, Viviana Arrigoni, Andrea Coletta, Irene Cannistraci, Svitlana Vyetenko, Paola Velardi, and Novella Bartolini (2023). “LOB-Based Deep Learning Models for Stock Price Trend Prediction: A Benchmark Study”. In: *arXiv preprint arXiv:2308.01915*.
- Rahimikia, Eghbal and Ser-Huang Poon (2021). “Machine Learning for Realised Volatility Forecasting”. In: *Available at SSRN 3707796*.
- Rush, Alexander M, Sumit Chopra, and Jason Weston (2015). “A Neural Attention Model for Abstractive Sentence Summarization”. In: *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics.
- Silver, David, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis (2017). “Mastering the Game of Go Without Human Knowledge”. In: *Nature* 550.7676, pp. 354–359.

- Sirignano, Justin A. (2019). “Deep Learning for Limit Order Books”. In: *Quantitative Finance* 19.4, pp. 549–570.
- Sirignano, Justin A. and Rama Cont (2019). “Universal Features of Price Formation in Financial Markets: Perspectives From Deep Learning”. In: *Quantitative Finance* 19.9, pp. 1449–1459.
- Sutskever, Ilya, Oriol Vinyals, and Quoc V. Le (2014). “Sequence to Sequence Learning with Neural Networks”. In: *Advances in Neural Information Processing Systems* 27.
- Tran, Dat Thanh, Alexandros Iosifidis, Juho Kanninen, and Moncef Gabbouj (2019). “Temporal Attention-Augmented Bilinear Network For Financial Time-Series Data Analysis”. In: *IEEE Transactions On Neural Networks And Learning Systems* 30.5, pp. 1407–1418.
- Tsantekidis, Avraam, Nikolaos Passalis, Anastasios Tefas, Juho Kanninen, Moncef Gabbouj, and Alexandros Iosifidis (2017). “Forecasting Stock Prices From The Limit Order Book Using Convolutional Neural Networks”. In: *2017 IEEE 19th Conference on Business Informatics (CBI)*. Vol. 1. IEEE, pp. 7–12.
- (2020). “Using Deep Learning For Price Prediction By Exploiting Stationary Limit Order Book Features”. In: *Applied Soft Computing* 93, p. 106401.
- Xu, Ke, Martin D. Gould, and Sam D. Howison (2018). “Multi-Level Order-Flow Imbalance in a Limit Order Book”. In: *Market Microstructure and Liquidity* 04.03n04, p. 1950011.
- Zhang, Zihao, Bryan Lim, and Stefan Zohren (2021). “Deep Learning For Market By Order Data”. In: *arXiv preprint arXiv:2102.08811*.
- Zhang, Zihao and Stefan Zohren (2021). “Multi-Horizon Forecasting For Limit Order Books: Novel Deep Learning Approaches And Hardware Acceleration Using Intelligent Processing Units”. In: *arXiv preprint arXiv:2105.10430*.
- Zhang, Zihao, Stefan Zohren, and Stephen Roberts (2018). “BDLOB: Bayesian Deep Convolutional Neural Networks For Limit Order Books”. In: *arXiv preprint arXiv:1811.10041*.
- (2019a). “DeepLOB: Deep Convolutional Neural Networks for Limit Order Books”. In: *IEEE Transactions On Signal Processing* 67.11, pp. 3001–3012.
- (2019b). “Extending Deep Learning Models For Limit Order Books To Quantile Regression”. In: *arXiv preprint arXiv:1906.04404*.